

COMP 3250

Data Analytics

Supervised Learning Cont.

© Dr. Abdulrauf Gidado

With examples and figures from Introduction to
Machine Learning with Python A Guide for Data
Scientists By Andreas C. Müller, Sarah Guido · 2016

Agenda

- Higher-Dimensional Datasets Linear Models
- Monday:
 - Ridge regression
 - Lasso
- Wednesday:
 - Linear models for classification
 - LogisticRegression,
 - Linear Support Vector Machines
 - Linear models for multiclass classification
 - Strengths, weaknesses, and parameters
 - Naive Bayes Classifiers



Linear Regression

- Let's take a look at how LinearRegression performs on a more complex dataset, like the Boston Housing dataset.
- This dataset has 506 samples and 105 derived features.

```
In [48]: X, y = mglearn.datasets.load_extended_boston()

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
lr = LinearRegression().fit(X_train, y_train)
```

```
In [49]: print("Training set score: {:.2f}".format(lr.score(X_train, y_train)))
print("Test set score: {:.2f}".format(lr.score(X_test, y_test)))
```

```
Training set score: 0.95
Test set score: 0.61
```

When comparing training set and test set scores, we find that we predict very accurately on the training set, but the R^2 on the test set is much worse.

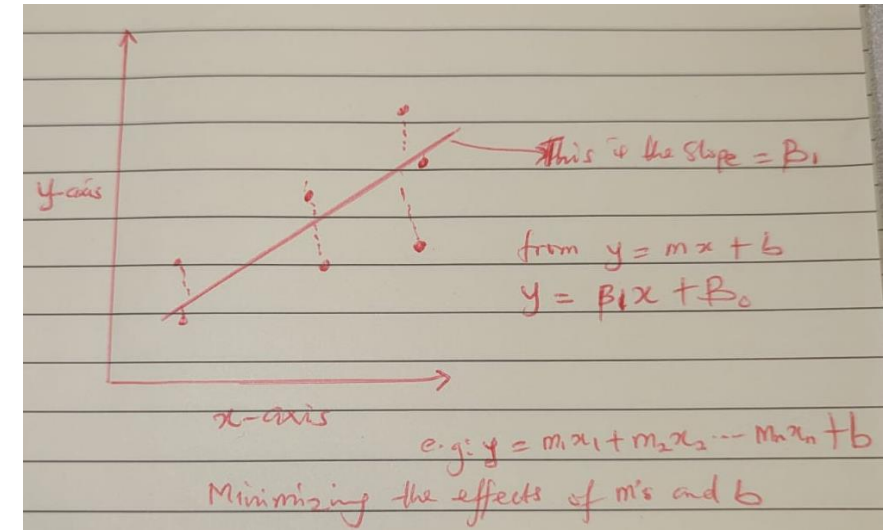


Why the discrepancy between the linear regression performance on the training set and the test set?

- This discrepancy between performance on the training set and the test set ***is a clear sign of overfitting***, and therefore we should try to find a model that allows us to control complexity. One of the most commonly used alternatives to standard linear regression is ***ridge regression***, which we will look into next.

Ridge regression

- Ridge regression is also a linear model for regression, so the formula it uses to make predictions is the same one used for ordinary least squares. In ridge regression, though, the coefficients (w) are chosen not only so that they predict well on the training data, but also to fit an additional constraint. We also want the magnitude of coefficients to be as small as possible; in other words, all entries of w should be **close to zero**. Intuitively, this means each feature should have as little effect on the outcome as possible (which translates to having a small slope), while still predicting well. This constraint is an example of what is called regularization. Regularization means explicitly restricting a model to avoid overfitting. The particular kind used by ridge regression is **known as L2 regularization**.



Let's get to action

Recall that on this dataset, with ordinary linear regression, we had a Training set score: 0.95 and Test set score: 0.61 (Overfitting!)

Ridge regression is implemented in *linear_model.Ridge*. Let's see how well it does on the extended Boston Housing dataset:



Ridge Regression

Ridge regression

```
In [52]: from sklearn.linear_model import Ridge

ridge = Ridge().fit(X_train, y_train)
print("Training set score: {:.2f}".format(ridge.score(X_train, y_train)))
print("Test set score: {:.2f}".format(ridge.score(X_test, y_test)))
```

Training set score: 0.89
Test set score: 0.75

- As you can see, the training set score of **Ridge** is lower than for **LinearRegression**, while the test set score is higher.
- This is **consistent** with our expectation because:
- With **linear regression**, we were overfitting our data. **Ridge** is a more restricted model, so we are **less likely** to overfit.
- A less complex model means worse performance on the training set, but better **generalization**!
- As we are only interested in generalization performance, we should choose the **Ridge model** over the **LinearRegression** model.

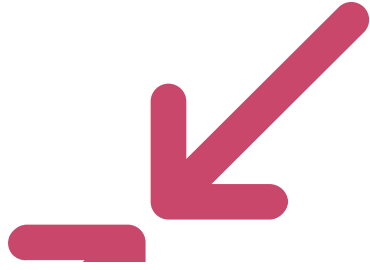
Ridge Regression (Cont.)

- How much importance the model places on *simplicity versus training set performance* can be specified by the user, using the **alpha** parameter.
- In the previous example, we used the *default parameter alpha=1.0*. There is no reason why this will give us the best trade-off, though.
- The optimum setting of alpha depends on the *particular dataset* we are using.
- *Increasing alpha forces coefficients* to move more toward zero, which decreases training set performance but might help generalization.

```
In [53]: ridge10 = Ridge(alpha=10).fit(X_train, y_train)
print("Training set score: {:.2f}".format(ridge10.score(X_train, y_train)))
print("Test set score: {:.2f}".format(ridge10.score(X_test, y_test)))
```

```
Training set score: 0.79
Test set score: 0.64
```

Decreasing alpha allows the coefficients to be less restricted,



```
In [54]: ridge01 = Ridge(alpha=0.1).fit(X_train, y_train)
print("Training set score: {:.2f}".format(ridge01.score(X_train, y_train)))
print("Test set score: {:.2f}".format(ridge01.score(X_test, y_test)))
```

```
Training set score: 0.93
Test set score: 0.77
```

Here, $\alpha=0.1$ seems to be working well. We could try decreasing alpha even more to improve generalization. For now, notice how the parameter alpha corresponds to the model complexity as shown in earlier weeks

For very *small values* of alpha, coefficients are barely restricted at all, and we end up with a model that resembles *LinearRegression*

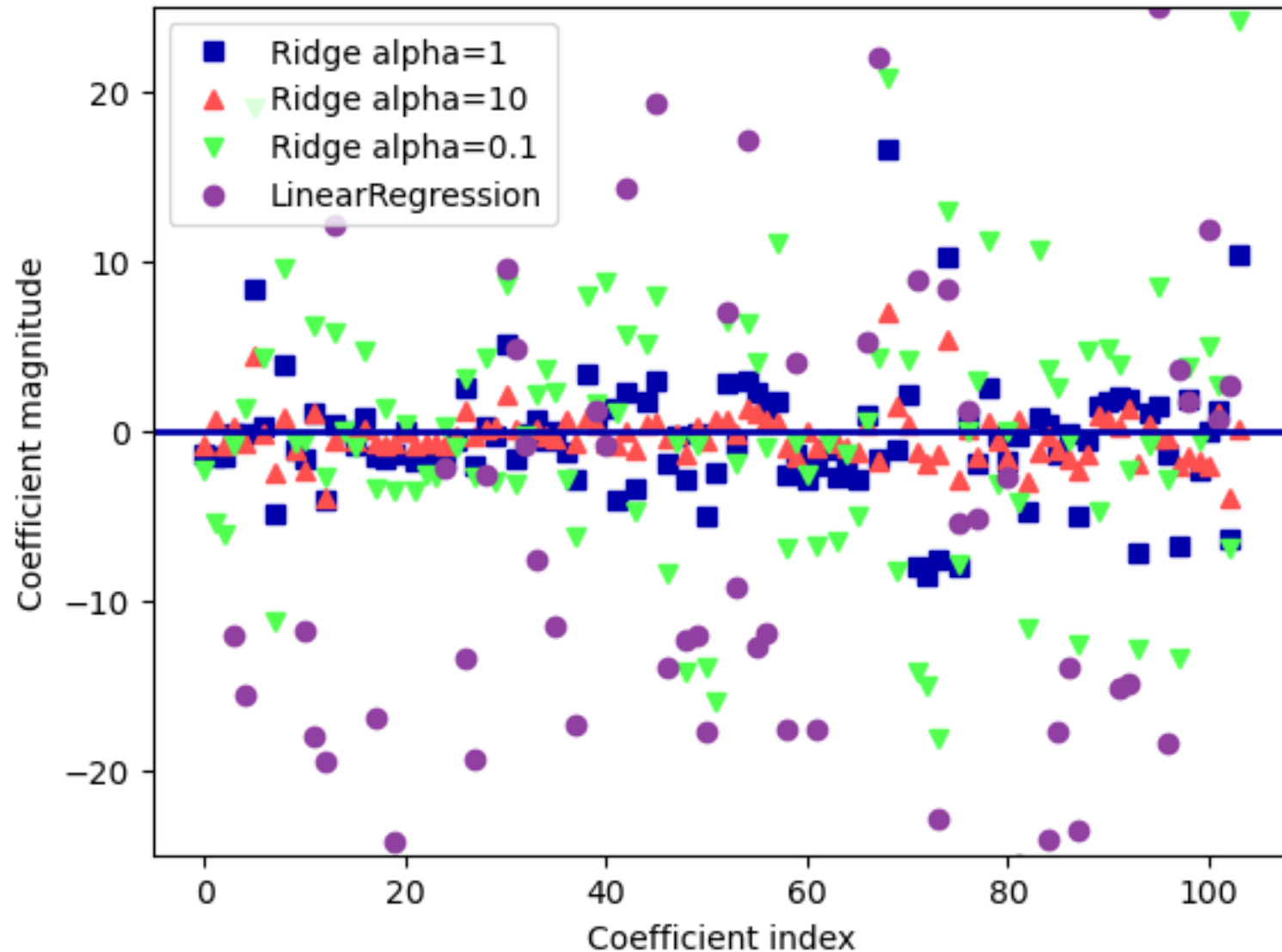
More qualitative insight into how the alpha parameter changes the model

We can also get a more qualitative insight into how the ***alpha*** parameter changes the model by inspecting the ***coef_*** attribute of models with different values of ***alpha***.

A higher alpha means a more restricted model, so we expect the entries of ***coef_*** to have smaller magnitude for a high value of ***alpha*** than for a ***low value of alpha***. This is confirmed in the plot below:

More qualitative insight into how the alpha parameter changes the model

- `plt.plot(ridge.coef_, 's', label="Ridge alpha=1")`
- `plt.plot(ridge10.coef_, '^', label="Ridge alpha=10")`
- `plt.plot(ridge01.coef_, 'v', label="Ridge alpha=0.1")`
- `plt.plot(lr.coef_, 'o', label="LinearRegression")`
- `plt.xlabel("Coefficient index")`
- `plt.ylabel("Coefficient magnitude")`
- `plt.hlines(0, 0, len(lr.coef_))`
- `plt.ylim(-25, 25)`
- `plt.legend()`



**More qualitative
insight into how
the alpha
parameter
changes the
model**

Comparing coefficient magnitudes for ridge regression with different values of alpha and linear regression

More qualitative insight into how the alpha parameter changes the model

Another way to understand the influence of regularization is to fix a value of ***alpha*** but vary the amount of training data available.

The test score for ridge is better, particularly for small subsets of the data.

For less than 400 data points, linear regression is not able to learn anything.

As more and more data becomes available to the model, both models improve, and linear regression catches up with ridge in the end.

The lesson here is that with enough training data, regularization becomes less important, and given enough data, ridge and linear regression will have the same performance (the fact that this happens here when using the full dataset is just by chance).

Lasso

An alternative to Ridge for regularizing linear regression is Lasso. As with ridge regression, using the lasso also restricts coefficients to be close to zero, but in a slightly different way, called L1 regularization.



The consequence of L1 regularization is that when using the lasso, some coefficients are exactly zero.



This means some features are entirely ignored by the model. This can be seen as a form of automatic feature selection.



Having some coefficients be exactly zero often makes a model easier to interpret, and can reveal the most important features of your model.



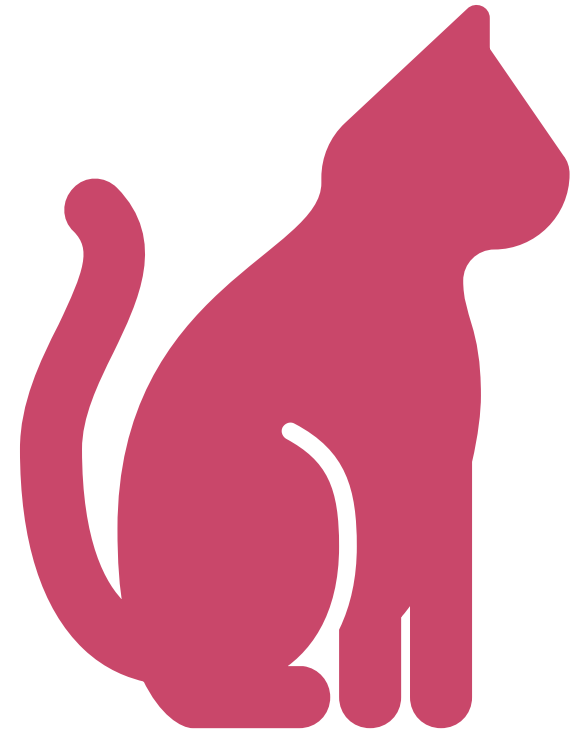
Let's see
Lasso
working

Let's see Lasso working

- `from sklearn.linear_model import Lasso`
- `lasso = Lasso().fit(X_train, y_train)`
- `print("Training set score:
{:.2f}".format(lasso.score(X_train, y_train)))`
- `print("Test set score:
{:.2f}".format(lasso.score(X_test, y_test)))`
- `print("Number of features used:
{:}".format(np.sum(lasso.coef_ != 0)))`

Output:

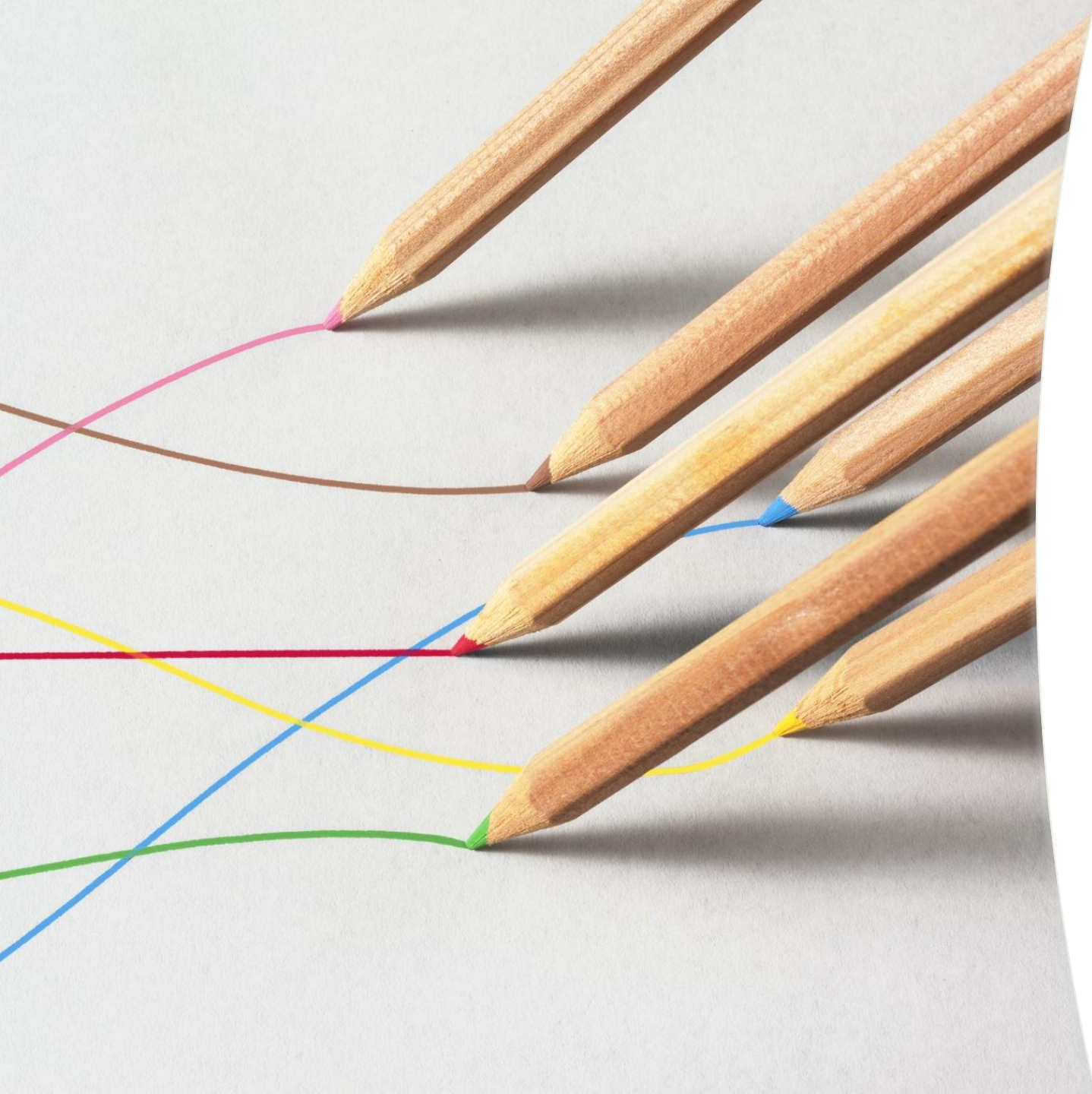
- Training set score: 0.29
- Test set score: 0.21
- Number of features used: 4





Bad moments

- Lasso just does the job badly on both training and test sets!
- Let's find out why?



Discussion

- This indicates that we are underfitting, and we find that it used only 4 of the 105 features.
- Similarly to Ridge, the Lasso also has a regularization parameter, α , that controls how strongly coefficients are pushed toward zero.

Solution

In the previous example, we used the default of $\alpha=1.0$. To reduce underfitting, let's try decreasing α .

When we do this, we also need to increase the default setting of `max_iter` (the maximum number of iterations to run)

Solution (Cont.)

```
# we increase the default setting of "max_iter",  
# otherwise the model would warn us that we should increase max_iter.  
lasso001 = Lasso(alpha=0.01, max_iter=100000).fit(X_train, y_train)  
print("Training set score: {:.2f}".format(lasso001.score(X_train,  
y_train)))  
print("Test set score: {:.2f}".format(lasso001.score(X_test, y_test)))  
print("Number of features used: {}".format(np.sum(lasso001.coef_ != 0)))
```

- Training set score: 0.90
- Test set score: 0.77
- Number of features used: 33

Solution



A lower alpha allowed us to fit a more complex model, which worked better on the training and test data.



The performance is slightly better than using Ridge, and we are using only 33 of the 105 features.



This makes this model potentially easier to understand.

Caution!: If we set alpha too low, however, we again remove the effect of regularization and end up overfitting, with a result similar to LinearRegression:

- Let's look at an example:

```
lasso00001 = Lasso(alpha=0.0001,  
max_iter=100000).fit(X_train, y_train)  
  
print("Training set score:  
{:.2f}".format(lasso00001.score(X_train,  
y_train)))  
  
print("Test set score:  
{:.2f}".format(lasso00001.score(X_test,  
y_test)))  
  
print("Number of features used:  
{:.2f}".format(np.sum(lasso00001.coef_ != 0)))
```

- **Output:**

- Training set score: 0.95
 - Test set score: 0.64
 - Number of features used: 94
-

Summary



The Ridge model with $\alpha=0.1$ has similar predictive performance as the lasso model with $\alpha=0.01$, but using Ridge, all coefficients are nonzero.



In practice, ridge regression is usually the first choice between these two models.



However, if you have a large amount of features and expect only a few of them to be important, Lasso might be a better choice.



Similarly, if you would like to have a model that is easy to interpret, Lasso will provide a model that is easier to understand, as it will select only a subset of the input features



scikit-learn also provides the ElasticNet class, which combines the penalties of Lasso and Ridge. In practice, this combination works best, though at the price of having two parameters to adjust: one for the L1 regularization, and one for the L2 regularization.